# EF Core: Pagination with Skip() & Take()

Concise lesson summary — database pagination, in-memory pitfalls, ordering, and highlighted key terms.

## 1. Core idea

**Pagination** loads data in smaller pages instead of loading the entire table. **Skip()** skips rows; **Take()** limits how many rows are returned.

**Key rule: pagination should happen in the database, not after loading all rows into memory.**

## 2. BAD approach: paginate in memory

```
var allStudents = context.Students.ToList();

var page = allStudents
    .OrderBy(s => s.StudentId)
    .Skip((pageNumber - 1) * pageSize)
    .Take(pageSize)
    .ToList();
```

• **ToList()** executes the query immediately and loads all students into memory.

• Skip() and Take() then run in C# after the database has returned every row.

**This causes unnecessary memory usage, extra data transfer, and worse performance as the table grows.**

## 3. GOOD approach: paginate in SQL Server

```
var page = context.Students
    .OrderBy(s => s.StudentId)
    .Skip((pageNumber - 1) * pageSize)
    .Take(pageSize)
    .ToList();
```

• EF Core translates this query so SQL Server returns only the requested page.

```
ORDER BY StudentId
OFFSET 5 ROWS FETCH NEXT 5 ROWS ONLY;
```

For page 2 with page size 5: **Skip = (2 - 1) × 5 = 5**, then **Take = 5**.

## 4. Always order before pagination

Apply **OrderBy()** before **Skip()** and **Take()**. Without a defined order, page contents can be inconsistent or unreliable.

```
int pageNumber = 2;
int pageSize = 5;

var query = context.Students
    .OrderBy(s => s.StudentId)
    .Skip((pageNumber - 1) * pageSize)
    .Take(pageSize);
```

## 5. SQL inspection

• **ToQueryString()** previews the SQL query shape before execution.

• **ToList()** executes the query and loads the requested page.

The lesson's SQL example uses **ORDER BY**, **OFFSET**, and **FETCH NEXT** for SQL Server pagination.

## 6. Common uses

Dashboards, APIs, admin tables, mobile apps, and search results use pagination to avoid retrieving huge datasets at once.

## Final takeaway

**OrderBy() → Skip() → Take() → ToList(). Keep Skip/Take before query execution so the database paginates instead of loading every row into application memory.**

Note: the SQL shown follows the lesson example; exact SQL may vary with query details and provider.